

Object Oriented Programming with C++

Subject Code: R23CP6404

Semester 3 · 5 Credits · CWIT R23

2 Units · 4 Lessons

Comprehensive Study Notes with Charts, Diagrams, Code Examples & Practice Quizzes

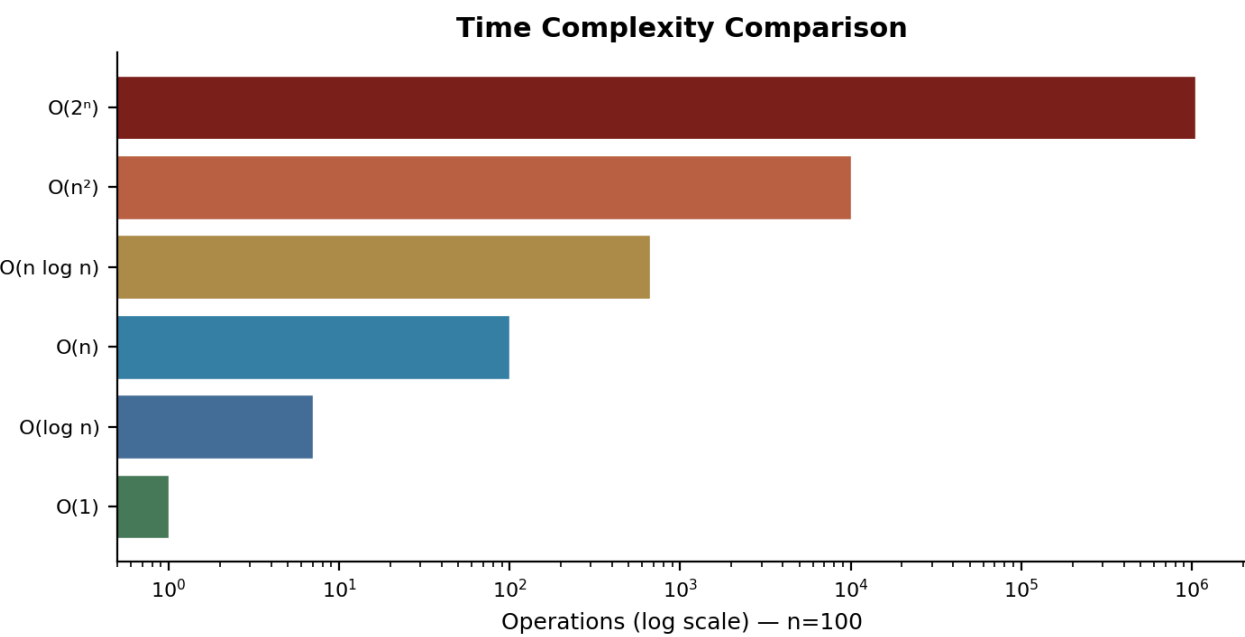
Lernio AI · Learning OS 2.0

Table of Contents

Visual Overview	3
Unit 1: Introduction to OOP and C++	4
OOP Concepts and Principles	4
Constructors and Destructors	7
Unit 2: Inheritance and Polymorphism	11
Inheritance — Code Reuse and Hierarchy	11
Polymorphism — Function Overloading and Virtual Functions	14
Subject Summary	18

Visual Overview

Key performance charts comparing the data structures and algorithms covered in this subject.



OOP Operation Complexity Comparison

Unit 1: Introduction to OOP and C++

Weightage: 15% · 2 lessons

OOP Concepts and Principles

Duration: 25 min · Difficulty: easy

Overview

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of 'objects' that contain data (attributes) and code (methods). C++ supports OOP through classes, objects, encapsulation, inheritance, and polymorphism. OOP makes code more modular, reusable, and easier to maintain.

Key Concepts

- **Class:** A blueprint/template that defines the structure (data members) and behavior (member functions) of objects
- **Object:** An instance of a class — a concrete entity created from the class blueprint
- **Encapsulation:** Bundling data and methods together, hiding internal details using access specifiers (private, public, protected)
- **Inheritance:** A class (child/derived) can inherit properties and methods from another class (parent/base), enabling code reuse
- **Polymorphism:** Same interface, different behavior — achieved through function overloading, operator overloading, and virtual functions
- **Abstraction:** Hiding complex implementation details and showing only essential features
- **Procedural vs OOP:** Procedural focuses on functions and steps. OOP focuses on objects and their interactions.

Formulas

Σ Access specifiers: public (accessible anywhere), private (class only), protected (class + derived classes)

Σ Class definition: `class ClassName { access: members; };` (note the semicolon)

Σ Object creation: `ClassName obj;` or `ClassName obj(params);`

Procedural vs Object-Oriented Programming

Feature	Procedural (C)	OOP (C++)
Focus	Functions and procedures	Objects and classes
Data	Global/local variables	Encapsulated in objects
Reusability	Function libraries	Inheritance + composition

Feature	Procedural (C)	OOP (C++)
Security	Data is often global	Private/protected access control
Example	C, Pascal	C++, Java, Python
Approach	Top-down	Bottom-up

OOP provides better data security and code reuse than procedural programming.

Class vs Object

CLASS (Blueprint)

```
class Car
+ brand: string
+ model: string
+ year: int
+ start()
+ stop()
```

→

OBJECTS (Instances)

```
car1: Car
brand = "Honda"
model = "City"
year = 2024
```

```
car2: Car
brand = "Tesla"
model = "M3"
year = 2025
```

Defining a Class and Creating Objects

```
#include <iostream>
#include <string>
using namespace std;

// Class definition
class Student {
private:
    string name;
    int rollNo;
    float marks;

public:
    // Member function to set data
    void setData(string n, int r, float m) {
        name = n;
        rollNo = r;
        marks = m;
    }

    // Member function to display data
    void display() {
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollNo << endl;
        cout << "Marks: " << marks << endl;
    }
};

int main() {
    // Create objects
    Student s1, s2;
```

```
s1.setData("Yash", 101, 85.5);
s2.setData("Riya", 102, 92.0);

s1.display();
cout << "---" << endl;
s2.display();

return 0;
}
```

The class `Student` has private data members (`name`, `rollNo`, `marks`) and public member functions (`setData`, `display`). Objects `s1` and `s2` are instances of `Student`, each with their own copy of the data.

Common Mistakes to Avoid

- ✗ Forgetting the semicolon after the class definition: `class Car { ... };` ← the `;` is required
- ✗ Accessing private members directly from outside the class — use public getter/setter functions instead
- ✗ Not using access specifiers — by default, members in a class are private (unlike struct where they're public)
- ✗ Confusing the class (blueprint) with objects (instances)

Exam Tips

- Always end a class definition with a semicolon: `};`
- Remember the 4 pillars: Encapsulation, Inheritance, Polymorphism, Abstraction
- By default, class members are private. struct members are public.
- Know the difference between class (blueprint) and object (instance)

Practice Questions

Q1. Which of the following is NOT one of the four pillars of OOP?

- A. Encapsulation
- B. Inheritance
- C. Compilation
- D. Polymorphism

Answer: C. The four pillars are: Encapsulation, Inheritance, Polymorphism, and Abstraction. Compilation is a process, not an OOP principle.

Q2. What is the default access specifier for members in a C++ class?

- A. public
- B. private
- C. protected

D. friend

Answer: B. In C++, class members are private by default. In a struct, members are public by default. This is the key difference between class and struct in C++.

Q3. What is the concept of hiding internal implementation details called?

A. Inheritance

B. Polymorphism

C. Encapsulation

D. Compilation

Answer: C. Encapsulation bundles data and methods together and hides internal details using access specifiers (private, protected). The user only sees the public interface.

Constructors and Destructors

Duration: 25 min · Difficulty: medium

Overview

A constructor is a special member function that is automatically called when an object is created. It initializes the object's data members. A destructor is called automatically when an object is destroyed (goes out of scope). Constructors have the same name as the class; destructors have a ~ prefix.

Key Concepts

- Constructor: same name as the class, no return type (not even void), called automatically when object is created
- Default constructor: takes no parameters — initializes members with default values
- Parameterized constructor: takes parameters to initialize members with specific values
- Copy constructor: initializes an object by copying another object of the same class
- Destructor: name starts with ~, called automatically when object is destroyed, used for cleanup (freeing memory, closing files)
- Constructor overloading: multiple constructors with different parameter lists
- Only ONE destructor per class — cannot be overloaded

Formulas

Σ Constructor: `ClassName() { /* init */ }`

Σ Parameterized: `ClassName(type param) { /* init */ }`

Σ Copy: `ClassName(const ClassName& other) { /* copy */ }`

```
Σ Destructor: ~ClassName() { /* cleanup */ }
```

Constructor vs Destructor

Feature	Constructor	Destructor
When called	Object creation	Object destruction
Name	Same as class name	~ + class name
Return type	None (not even void)	None
Parameters	Can have parameters	No parameters
Overloading	Yes (multiple)	No (only one)
Purpose	Initialize members	Cleanup resources

Both are called automatically — you never call them directly.

All Types of Constructors + Destructor

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    int length, width;

public:
    // 1. Default constructor
    Rectangle() {
        length = 0;
        width = 0;
        cout << "Default constructor called" << endl;
    }

    // 2. Parameterized constructor
    Rectangle(int l, int w) {
        length = l;
        width = w;
        cout << "Parameterized constructor called" << endl;
    }

    // 3. Copy constructor
    Rectangle(const Rectangle& other) {
        length = other.length;
        width = other.width;
        cout << "Copy constructor called" << endl;
    }

    // Destructor
    ~Rectangle() {
        cout << "Destructor called for (" << length << ", " << width << ")" << endl;
    }

    int area() { return length * width; }
};
```



```

int main() {
    Rectangle r1; // default constructor
    Rectangle r2(5, 3); // parameterized constructor
    Rectangle r3 = r2; // copy constructor

    cout << "r1 area: " << r1.area() << endl; // 0
    cout << "r2 area: " << r2.area() << endl; // 15
    cout << "r3 area: " << r3.area() << endl; // 15

    // Destructors called in reverse order: r3, r2, r1
    return 0;
}

```

Three constructors are demonstrated. The destructor is called automatically when each object goes out of scope at the end of main(). Objects are destroyed in reverse order of creation (LIFO).

Common Mistakes to Avoid

- ✗ Giving a constructor a return type (even void) — this makes it a regular function, not a constructor
- ✗ Forgetting the ~ in the destructor name
- ✗ Trying to call a constructor directly — it's called automatically
- ✗ Not initializing all members in the constructor (uninitialized members have garbage values)
- ✗ Confusing the copy constructor with the assignment operator (=)

Exam Tips

- Constructors have NO return type — not even void
- Destructors are called in reverse order of construction (LIFO)
- If you don't define any constructor, the compiler provides a default constructor
- If you define a parameterized constructor, the compiler does NOT provide a default constructor
- Know the three types: default, parameterized, copy

Practice Questions

Q1. What is the name of a destructor for class 'Car'?

- A. Car()
- B. ~Car()**
- C. destroy()
- D. Car~

Answer: B. A destructor is named with a tilde (~) prefix followed by the class name: ~Car(). It takes no parameters and has no return type.

Q2. What happens if you define a parameterized constructor but no default constructor?

- A.** Compiler provides a default constructor automatically
- B.** You cannot create objects without parameters
- C.** Compiler gives an error
- D.** Objects are created with null values

Answer: B. If you define any constructor, the compiler does NOT provide a default constructor. So you can only create objects using the parameterized constructor. To create objects without parameters, you must explicitly define a default constructor.

Q3. In what order are destructors called when multiple objects go out of scope?

- A.** Same order as construction
- B.** Reverse order of construction
- C.** Random order
- D.** Alphabetical order

Answer: B. Destructors are called in reverse order of construction (LIFO — Last In, First Out). If objects were created as r1, r2, r3, destructors are called as r3, r2, r1.

Unit 2: Inheritance and Polymorphism

Weightage: 25% · 2 lessons

Inheritance — Code Reuse and Hierarchy

Duration: 30 min · Difficulty: medium

Overview

Inheritance is a mechanism where a new class (derived/child class) acquires the properties and behaviors of an existing class (base/parent class). It enables code reuse, hierarchical classification, and extensibility. The derived class can add new members or override existing ones.

Key Concepts

- Base class (parent): the class being inherited from
- Derived class (child): the class that inherits from the base class
- Single inheritance: one base class, one derived class
- Multiple inheritance: a class inherits from multiple base classes
- Multilevel inheritance: $A \rightarrow B \rightarrow C$ (chain of inheritance)
- Hierarchical inheritance: multiple classes inherit from one base class
- Access modes: public inheritance (public members stay public), protected inheritance, private inheritance
- The derived class has access to public and protected members of the base class, but NOT private members

Formulas

Σ Syntax: `class Derived : access_mode Base { /* ... */ }`

Σ Public inheritance: `class Student : public Person { }`

Σ Constructor call order: Base constructor → Derived constructor

Σ Destructor call order: Derived destructor → Base destructor

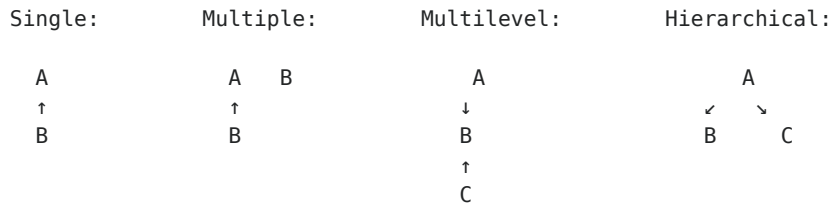
Inheritance Access Modes

Base Member	Public Inheritance	Protected Inheritance	Private Inheritance
public	public in derived	protected in derived	private in derived
protected	protected in derived	protected in derived	private in derived

Base Member	Public Inheritance	Protected Inheritance	Private Inheritance
private	NOT accessible	NOT accessible	NOT accessible

Private members of the base class are NEVER directly accessible in the derived class, regardless of inheritance mode.

Types of Inheritance



A = Base class, B/C = Derived class

Single Inheritance — Person → Student

```
#include <iostream>
#include <string>
using namespace std;

// Base class
class Person {
protected: // accessible in derived classes
string name;
int age;

public:
void setPerson(string n, int a) {
name = n;
age = a;
}
void displayPerson() {
cout << "Name: " << name << ", Age: " << age;
}
};

// Derived class – inherits from Person
class Student : public Person {
private:
int rollNo;
float marks;

public:
void setStudent(string n, int a, int r, float m) {
setPerson(n, a); // call base class method
rollNo = r;
marks = m;
}

void displayStudent() {
displayPerson(); // call base class method
cout << ", Roll: " << rollNo << ", Marks: " << marks << endl;
}
};
```

```
int main() {
    Student s;
    s.setStudent("Yash", 20, 101, 85.5);
    s.displayStudent();
    // Output: Name: Yash, Age: 20, Roll: 101, Marks: 85.5
    return 0;
}
```

Student inherits from Person. It can access the protected members (name, age) and public methods (setPerson, displayPerson) of Person. Student adds its own members (rollNo, marks) and methods (setStudent, displayStudent).

Common Mistakes to Avoid

- ✗ Forgetting the access mode: `class B : A` defaults to private inheritance in C++
- ✗ Trying to access private members of the base class directly — use protected instead
- ✗ Forgetting to call the base class constructor in the derived class constructor's initializer list
- ✗ Confusing 'is-a' (inheritance) with 'has-a' (composition) relationships

Exam Tips

- Use 'protected' for members that should be accessible in derived classes but not outside
- Constructor order: Base first, then Derived. Destructor order: Derived first, then Base.
- Know the 5 types of inheritance: single, multiple, multilevel, hierarchical, hybrid
- Private members are NEVER directly accessible in derived classes — use protected

Practice Questions

Q1. If class B inherits from class A privately, what happens to A's public members in B?

- A. They remain public in B
- B. They become private in B**
- C. They become protected in B
- D. They are not accessible at all

Answer: B. With private inheritance, all public and protected members of A become private in B. This means they're accessible inside B's methods but not from outside or in further derived classes.

Q2. What is the order of constructor calls in inheritance?

- A. Derived → Base
- B. Base → Derived**
- C. Random
- D. Only Derived constructor is called

Answer: B. The base class constructor is called first (to initialize the base part), then the derived class constructor is called (to initialize the derived part). Think of it as building the foundation before the house.

Q3. Which access specifier allows members to be accessed in derived classes but not from outside?

- A. public
- B. private
- C. protected
- D. friend

Answer: C. Protected members are accessible within the class and in derived classes, but NOT from outside the class. This is specifically designed for inheritance scenarios.

Polymorphism — Function Overloading and Virtual Functions

Duration: 30 min · Difficulty: hard

Overview

Polymorphism means 'many forms' — the same function name or operator can behave differently depending on the context. In C++, polymorphism is achieved through function overloading (compile-time), operator overloading (compile-time), and virtual functions (runtime). Runtime polymorphism uses a base class pointer to call derived class methods.

Key Concepts

- Compile-time polymorphism: resolved at compile time — function overloading and operator overloading
- Runtime polymorphism: resolved at runtime — virtual functions and overriding
- Function Overloading: same function name, different parameter lists (number, type, or order)
- Operator Overloading: redefine what an operator (+, -, [], etc.) does for user-defined types
- Virtual Function: declared with 'virtual' keyword in base class, overridden in derived class
- Override: derived class provides its own implementation of a virtual function
- Base class pointer + virtual function = runtime polymorphism (dynamic dispatch)
- Pure Virtual Function: virtual void func() = 0; — makes the class abstract (cannot be instantiated)

Formulas

Σ Virtual: virtual returnType funcName(params) { }

Σ Pure virtual: virtual returnType funcName(params) = 0;

Σ Function overloading: void print(int); void print(float); void print(string);

Compile-time vs Runtime Polymorphism

Feature	Compile-time	Runtime
Mechanism	Function/operator overloading	Virtual functions
Binding	Early binding (at compile)	Late binding (at runtime)
Speed	Faster (resolved at compile)	Slightly slower (vtable lookup)
Flexibility	Less flexible	More flexible
Example	void print(int), void print(char)	virtual void draw()

Runtime polymorphism uses a vtable (virtual table) to look up the correct function at runtime.

Runtime Polymorphism with Virtual Functions

```
#include <iostream>
using namespace std;

// Base class
class Shape {
public:
    virtual void draw() { // virtual function
        cout << "Drawing a generic shape" << endl;
    }
    virtual double area() = 0; // pure virtual – abstract class
};

class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}
    void draw() override { // override base class virtual
        cout << "Drawing a circle" << endl;
    }
    double area() override {
        return 3.14159 * radius * radius;
    }
};

class Rectangle : public Shape {
    double length, width;
public:
    Rectangle(double l, double w) : length(l), width(w) {}
    void draw() override {
        cout << "Drawing a rectangle" << endl;
    }
    double area() override {
        return length * width;
    }
};

int main() {
    // Base class pointers pointing to derived class objects
    Shape* s1 = new Circle(5.0);
```

```

Shape* s2 = new Rectangle(4.0, 6.0);

s1->draw(); // Output: Drawing a circle (runtime dispatch!)
cout << "Area: " << s1->area() << endl; // 78.54

s2->draw(); // Output: Drawing a rectangle
cout << "Area: " << s2->area() << endl; // 24.0

delete s1;
delete s2;
return 0;
}

```

Even though *s1* and *s2* are *Shape* pointers, the correct *draw()* and *area()* methods of *Circle* and *Rectangle* are called at runtime because they are declared *virtual*. This is runtime polymorphism. *Shape* is abstract (has a pure virtual function *area()*=0), so you can't create a *Shape* object directly.

Common Mistakes to Avoid

- ✗ Forgetting the 'virtual' keyword in the base class — without it, the base class method is called even for derived objects
- ✗ Not making the base class destructor virtual — when deleting through a base pointer, the derived destructor won't be called
- ✗ Confusing overloading (same name, different params) with overriding (same name, same params, different class)
- ✗ Trying to instantiate an abstract class (one with pure virtual functions)

Exam Tips

- Virtual functions enable runtime polymorphism — always use 'virtual' in the base class
- Pure virtual function (= 0) makes the class abstract — cannot be instantiated
- Function overloading is compile-time; virtual functions are runtime
- Always make the base class destructor virtual if you use inheritance + dynamic allocation
- Know the difference: overloading (same name, different params) vs overriding (same name, same params, derived class)

Practice Questions

Q1. What keyword is used to achieve runtime polymorphism in C++?

- A. static
- B. virtual**
- C. override
- D. inline

Answer: B. The 'virtual' keyword tells the compiler to use dynamic dispatch (vtable lookup) at runtime. Without it, the base class method is called based on the pointer type, not the actual object type.

Q2. What is a pure virtual function?

- A. A function with no parameters
- B. A function declared with = 0, making the class abstract**
- C. A function that returns 0
- D. A function that cannot be overridden

Answer: B. A pure virtual function is declared with '= 0' syntax: virtual void func() = 0;. It has no implementation in the base class and must be overridden in derived classes. A class with at least one pure virtual function is abstract and cannot be instantiated.

Q3. What is function overloading?

- A. Redefining a function in a derived class
- B. Having multiple functions with the same name but different parameters**
- C. Making a function virtual
- D. Calling a function from another function

Answer: B. Function overloading is having multiple functions with the same name but different parameter lists (different number, type, or order of parameters). The compiler decides which version to call based on the arguments. This is compile-time polymorphism.

Subject Summary

This PDF contains 4 lessons across 2 units with 12 practice questions.

Use these notes alongside the curated YouTube lectures on the Lernio platform. Watch the videos, read the notes, practise the quizzes, and use Ask LEO for any doubts.

Lernio AI · Learning OS 2.0 · CWIT R23